BÀI THỰC HÀNH BUỔI 2

Express, route, controller và middleware

Thời lượng gợi ý: 120 phút. Yêu cầu trước: đã biết chạy node file.js và request/response của buổi 1. Công cụ: Node.js, npm, VS Code, trình duyệt; cài Express qua npm (cần kết nối mạng khi cài lần đầu).

Kết quả cần đạt: Tạo dự án Node, chạy Express; trả JSON từ GET /students; chuyển route và controller thành file riêng; dùng middleware ghi log; đọc tham số :id và trả status đúng. Dữ liệu mảng chỉ là ví dụ, chưa kết nối SQL Server.

Nộp bài: Thư mục dự án gồm package.json, app.js, routes, controllers, middlewares; không nộp node_modules. Kèm ảnh ba kết quả /students, /students/1, /students/10 và log terminal. Mã mẫu có chú thích trong ZIP.

Thực hành 2.1 Khởi tạo dự án 10 phút

Mở terminal và chạy trong vị trí muốn lưu bài:

```
mkdir backend-buoi2
cd backend-buoi2
npm init -y
```

Mở package.json: name là tên gói, version là phiên bản, scripts là lệnh viết tắt, dependencies sẽ ghi thư viện cài vào. node_modules là thư viện được tải về, không phải mã do sinh viên tự viết. Trên một số máy Windows, nếu mkdir báo đã tồn tại, vào đúng thư mục đã tạo.

Thực hành 2.2 Cài Express, tạo server 20 phút

```
npm install express
```

Tạo app.js:

```
const express = require("express"); // Nạp thư viện Express.
const app = express();              // Tạo ứng dụng.
app.get("/", (req, res) => {
  res.send("Hello Express!");       // Response ở đường dẫn /.
});
app.listen(3000, () => console.log("Server: http://localhost:3000"));
```

Chạy node app.js, mở http://localhost:3000/. So với buổi 1, Express cung cấp app.get và res.send để ta đỡ phải tự rẽ nhánh HTTP cơ bản. GET là phương thức yêu cầu đọc dữ liệu; / là đường dẫn. Dừng bằng Ctrl+C trước khi chạy bản mã ở bước sau.

Thực hành 2.3 API GET đầu tiên 15 phút

Trong app.js, thêm **trước** app.listen:

```
app.get("/students", (req, res) => {
  const students = [
    { id: 1, name: "Nguyen Van A" },
    { id: 2, name: "Tran Van B" }
  ];
  res.json(students); // Trả mảng dưới dạng JSON.
});
```

Chạy lại node app.js; mở http://localhost:3000/students. Hỏi bạn cùng bàn: GET /students gồm phương thức nào, URL nào, nội dung trả về là gì? Lệnh res.json() gửi JSON và header phù hợp. Chỉ tải lại trang không làm code đang chạy tự cập nhật.

Thực hành 2.4 Tách route 20 phút

Tạo routes/student.routes.js:

```
const express = require("express");
const router = express.Router(); // Gom các đường dẫn cùng chủ đề.
router.get("/", (req, res) => res.json([
  { id: 1, name: "Nguyen Van A" },
  { id: 2, name: "Tran Van B" }
]));
module.exports = router; // Cho file khác require được router.
```

Trong app.js, xóa đoạn app.get("/students", ...), thêm:

```
const studentRoutes = require("./routes/student.routes");
app.use("/students", studentRoutes); // /students + / = /students.
```

Đặt app.use trước app.listen. Chạy lại và so sánh JSON: kết quả giữ nguyên. require đi từ thư mục của file gọi; kiểm tra dấu ./ và tên file khi thấy Cannot find module.

Thực hành 2.5 Tách controller 20 phút

Tạo controllers/student.controller.js:

```
const students = [
  { id: 1, name: "Nguyen Van A" },
  { id: 2, name: "Tran Van B" }
];
function getStudents(req, res) {
  return res.json(students); // Nơi xử lý yêu cầu.
}
module.exports = { getStudents };
```

Thay nội dung routes/student.routes.js:

```
const express = require("express");
const controller = require("../controllers/student.controller");
const router = express.Router();
router.get("/", controller.getStudents); // Chỉ nối URL với hàm.
module.exports = router;
```

Chạy lại, thử /students. Route chọn đúng hàm; controller quyết định nội dung trả về. Tách file giúp dễ tìm trách nhiệm, không làm dữ liệu tự động chuyển sang database.

Thực hành 2.6 Middleware ghi log 15 phút

Tạo middlewares/logger.middleware.js:

```
function logger(req, res, next) {
  console.log("Method:", req.method, "URL:", req.originalUrl);
  next(); // Cho request đi tới route kế tiếp.
}
module.exports = logger;
```

Trong app.js, thêm trước app.use("/students", studentRoutes):

```
const logger = require("./middlewares/logger.middleware");
app.use(logger);
```

Chạy lại, mở /students và xem terminal. next() chuyển quyền xử lý; nếu bỏ next() mà cũng không gửi response, trình duyệt sẽ chờ mãi. Khôi phục dòng đó sau khi quan sát. Buổi sau middleware có thể kiểm tra quyền, còn ở đây chỉ ghi log.

Thử thách Tra cứu theo ID 15 phút

Trong controller, bổ sung hàm (và sửa module.exports thành { getStudents, getStudentById }):

```
function getStudentById(req, res) {
  const rawId = req.params.id; // Giá trị trên URL là chuỗi.
  if (!/^[1-9]\d*$/.test(rawId)) {
    return res.status(400).json({ message: "ID phải là số nguyên dương" });
  }
  const student = students.find(item => item.id === Number(rawId));
  if (!student) return res.status(404).json({ message: "Không tìm thấy sinh viên" });
  return res.json(student);
}
```

Trong route, sau route /, thêm:

```
router.get("/:id", controller.getStudentById);
```

Thử /students/1 (200, sinh viên 1), /students/2 (200, sinh viên 2), /students/10 (404), /students/abc (400). Chú ý: ví dụ ban đầu trong file ý tưởng luôn trả cùng một người cho mọi ID; bản này tìm trong mảng nên kết quả phản ánh URL thực sự.

Chốt bài và xử lý lỗi 5 phút

Tự kể lại: Browser → middleware logger → route /students/:id → controller → JSON response. Nếu thấy EADDRINUSE, đóng server cũ bằng Ctrl+C. Nếu thiếu module, chạy npm install tại đúng dự án. Nếu route không nhận, kiểm tra app.use đã gắn và thứ tự middleware. Nếu trình duyệt quay mãi, kiểm tra next() hoặc res.send/res.json.

Tự đánh giá: Chỉ ra trong code vị trí nhận URL; nơi chọn sinh viên; nơi in method; nơi trả status 404. Giải thích tại sao thay controller mà không cần đổi URL.

Nguồn nội bộ đã đối chiếu: File ý tưởng Đã dán markdown (1).md; slide bài 01; tài liệu học phần Backend; các lab 4–7 để liên thông kiến thức route, controller, middleware với SQL Server/JWT về sau.